23DSCP206-Data Mining Using R

1. Simple Calculator

Aim

To write an R script to create a **Simple Calculator** that performs basic arithmetic operations: addition, subtraction, multiplication, and division, using functions and control structures.

Procedure

1. **Define Functions**: Create separate functions for each arithmetic operation (addition, subtraction, multiplication, division).
2. **User Input**: Use the `readline()` function to accept input for the two numbers and the operation choice.
3. **Display Menu**: Show a menu with operation options and prompt the user to select one.
4. **Decision Making**: Use conditional statements (`if-else`) to call the appropriate function based on the user's choice.
5. **Loop for Continuation**: Use a `while` loop to allow the user to perform multiple calculations until they choose to exit.
6. **Handle Division**: Ensure division is handled properly by checking for division by zero if needed.

Program:

```
# Define functions for arithmetic operations
add <- function(n1, n2) {
  print(paste(n1, "+", n2, "=", n1 + n2))
}
subtract <- function(n1, n2) {
  print(paste(n1, "-", n2, "=", n1 - n2))
}
multiply <- function(n1, n2) {
  print(paste(n1, "*", n2, "=", n1 * n2))
}
divide <- function(n1, n2) {
  if (n2 != 0) {
    print(paste(n1, "/", n2, "=", n1 / n2))
  } else {
    print("Error: Division by zero is not allowed.")
  }
}

# Display program header
print("*** Simple Calculator ***")
print("----------------------------")
ch <- 'y' # Initialize user choice variable

# Start the loop
while (ch == 'y' | ch == 'Y') {
  # Input two numbers
  n1 <- as.integer(readline(prompt = "Enter the value for n1: "))
  n2 <- as.integer(readline(prompt = "Enter the value for n2: "))

  # Display operation menu
  print("1. Addition")
```

```
  print("2. Subtraction")
  print("3. Multiplication")
  print("4. Division")

  # Accept operation choice
  op <- as.integer(readline(prompt = "Enter the operation number (1/2/3/4): "))

  # Perform the selected operation
  if (op == 1) {
    add(n1, n2)
  } else if (op == 2) {
    subtract(n1, n2)
  } else if (op == 3) {
    multiply(n1, n2)
  } else if (op == 4) {
    divide(n1, n2)
  } else {
    print("Invalid operation. Please enter a valid choice.")
  }

  # Prompt for continuation
  print("Do you want to continue?")
  ch <- readline(prompt = "Enter y / n: ")
}
```

Output:

```
*** Simple Calculator ***
-----------------------------
Enter the value for n1: 12
Enter the value for n2: 8
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter the operation number (1/2/3/4): 1
12 + 8 = 20
Do you want to continue?
Enter y / n: y


Enter the value for n1: 10
Enter the value for n2: 0
Enter the operation number (1/2/3/4): 4
Error: Division by zero is not allowed.

Do you want to continue?
Enter y / n: n
```

Result

The program successfully implements a Simple Calculator in R, demonstrating using functions, conditional statements, and loops. Users can perform multiple calculations until they choose to exit.

2. Understanding Vector Concepts in R

Aim

To explore and perform various operations and manipulations using vectors in R, including creation, indexing, modification, operations, and handling missing values.

Procedure

1. Create Vectors: Learn to create numeric, character, and logical vectors. Explore sequences and replications.
2. Access and Index Elements: Access vector elements using positive, negative, and logical indexing.
3. Modify Vectors: Update existing elements, add new elements, and extend vectors.
4. Perform Vector Operations: Apply arithmetic, scalar, and logical operations on vectors.
5. Use Built-in Functions: Utilize functions like `sum()`, `mean()`, and `sort()` to analyze vectors.
6. Understand Recycling: Observe how shorter vectors are recycled during operations.
7. Filter Vector Elements: Extract elements that satisfy certain conditions.
8. Combine Vectors: Combine multiple vectors into one.
9. Handle Missing Values: Identify and handle missing values using `is.na()` and `na.omit()`.
10. Apply Functions to Vectors: Use functions to manipulate vector elements.
11. Understand Coercion: Explore how R coerces elements of different types in a vector.

Program:

```
# 1. Creating Vectors
numeric_vec <- c(10, 20, 30, 40)
char_vec <- c("R", "Programming", "Vector")
logical_vec <- c(TRUE, FALSE, TRUE, FALSE)
seq_vec <- seq(1, 10, by = 2)
rep_vec <- rep(5, times = 4)

# 2. Accessing and Indexing
first_element <- numeric_vec[1]
subset_elements <- numeric_vec[2:4]
specific_elements <- numeric_vec[c(1, 3)]
exclude_element <- numeric_vec[-2]
logical_indexing <- numeric_vec[logical_vec]

# 3. Modifying Vectors
numeric_vec[2] <- 25
numeric_vec <- c(numeric_vec, 50)

# 4. Vector Operations
vec_a <- c(1, 2, 3)
vec_b <- c(4, 5, 6)
addition <- vec_a + vec_b
multiplication <- vec_a * vec_b
scalar_multiplication <- vec_a * 2
logical_comparison <- vec_a > 2
```

```
# 5. Vector Functions
vector_sum <- sum(numeric_vec)
vector_mean <- mean(numeric_vec)
vector_length <- length(numeric_vec)
sorted_vector <- sort(numeric_vec)

# 6. Vector Recycling
vec_short <- c(1, 2)
recycled_operation <- vec_a + vec_short

# 7. Filtering Vectors
filtered_vec <- numeric_vec[numeric_vec > 20]

# 8. Combining Vectors
combined_vec <- c(vec_a, vec_b)

# 9. Handling Missing Values (NA)
vec_with_na <- c(1, 2, NA, 4)
na_removed <- na.omit(vec_with_na)
na_check <- is.na(vec_with_na)

# 10. Apply Function to Each Element
sqrt_vec <- sqrt(numeric_vec)

# 11. Coercion in Vectors
mixed_vec <- c(1, "two", TRUE)
```

Result:

The program demonstrates successful creation, manipulation, and operations on vectors, including indexing, filtering, and handling missing values. Additionally, vector-specific functions, recycling, and coercion are effectively applied and observed.

3. Understanding Lists in R

Aim: To understand the creation, manipulation, and various operations on lists in R, including accessing elements, modifying components, combining lists, using nested lists, and applying functions on list elements.

Procedure:

1. **Create Lists:** Define a list containing various data types such as numeric, character, and logical vectors.
2. **Access Elements:** Access components of the list using index, name, or specific indices within a component.
3. **Modify Elements:** Update components, add new components, and remove existing ones.
4. **Combine Lists:** Merge two lists into a single list.
5. **Apply Functions:** Use `lapply` and `sapply` to apply functions across list components.
6. **Work with Nested Lists:** Create and access nested lists.
7. **Unlist:** Flatten a list component into a simple vector.
8. **Check and Iterate:** Inspect the structure of a list using `str()` and iterate over its components.
9. **Use Cases:** Extract specific components and perform summary statistics.

Program:

```
# 1. Creating Lists
list_example <- list(
  numbers = c(1, 2, 3, 4),                # Numeric vector
  words = c("apple", "banana", "cherry"),    # Character vector
  logicals = c(TRUE, FALSE, TRUE)           # Logical vector
)
print("Original List:")
print(list_example)

# 2. Accessing List Elements
# By index
print("Access by index (first element):")
print(list_example[[1]])

# By name
print("Access by name (words):")
print(list_example$words)

# Access specific element in a component
print("Access specific value in words (second element):")
print(list_example$words[2])

# 3. Modifying List Elements
list_example$numbers <- c(10, 20, 30)  # Replace the numeric vector
print("Modified List (numbers replaced):")
print(list_example)

# Adding new components
list_example$new_component <- "Hello, R!"  # Add a new component
```

```
print("List after adding a new component:")
print(list_example)

# 4. Removing List Elements
list_example$new_component <- NULL  # Remove the new component
print("List after removing the new component:")
print(list_example)

# 5. Combining Lists
list_combined <- c(list_example, list(additional = c(100, 200)))
print("Combined List:")
print(list_combined)

# 6. Applying Functions on Lists
# Using lapply to apply a function to each element
print("Length of each component in the list:")
print(lapply(list_example, length))

# Using sapply to simplify the result
print("Length of each component (simplified):")
print(sapply(list_example, length))

# 7. Nested Lists
nested_list <- list(
  first = list(a = 1, b = 2),
  second = list(c = "hello", d = "world")
)
print("Nested List:")
print(nested_list)

# Accessing elements in nested lists
print("Access nested element (second$c):")
print(nested_list$second$c)

# 8. Unlisting
flat_vector <- unlist(list_example$numbers)
print("Unlisted numeric vector:")
print(flat_vector)

# 9. Checking and Iterating Over a List
print("Checking structure of the list:")
str(list_example)

print("Iterating over the list:")
for (item in list_example) {
  print(item)
}
```

```
# 10. Use Cases
# Extracting specific components
selected_components <- list_example[c("numbers", "words")]
print("Selected components (numbers and words):")
print(selected_components)

# Summary of the numbers in the list
print("Summary of the numbers component:")
print(summary(list_example$numbers))
```

Result:

The R script successfully demonstrates various operations on lists, including creation, access, modification, combination, nesting, unlisting, and applying functions. Additionally, it showcases the utility of lists in handling heterogeneous data and complex structures effectively.

4. Matrix Operations in R

Aim:

To understand the creation, manipulation, and operations on matrices in R programming.

Procedure:

1. Creating Matrices:
   - Use the `matrix()` function to create matrices, filling column-wise or row-wise.
   - Combine vectors into matrices using `rbind()` and `cbind()`.
2. Accessing Matrix Elements:
   - Access specific elements, rows, or columns using indexing.
3. Modifying Matrices:
   - Update elements or rows in a matrix using indexing.
4. Matrix Arithmetic:
   - Perform element-wise addition, scalar multiplication, and matrix multiplication.
5. Matrix Functions:
   - Calculate transpose, row sums, column sums, row means, and column means.
6. Logical Operations:
   - Apply conditions to matrices and return logical matrices.
7. Binding Matrices:
   - Combine matrices by columns or rows using `cbind()` and `rbind()`.
8. Diagonal and Identity Matrices:
   - Create identity matrices using `diag()` and customize diagonal elements.
9. Matrix Subsetting:
   - Extract submatrices using row and column indexing.
10. Matrix Operations:
    - Calculate the determinant and inverse of matrices.
11. Checking Properties:
    - Use functions like `dim()`, `nrow()`, and `ncol()` to check matrix properties.

Program:

```
# 1. Creating Matrices
matrix1 <- matrix(1:9, nrow = 3, ncol = 3)
print("Matrix 1 (3x3 filled column-wise):")
print(matrix1)

matrix2 <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
print("Matrix 2 (2x3 filled row-wise):")
print(matrix2)

row1 <- c(1, 2, 3)
row2 <- c(4, 5, 6)
matrix3 <- rbind(row1, row2)
print("Matrix 3 (using rbind):")
print(matrix3)

col1 <- c(1, 4)
col2 <- c(2, 5)
```

```
matrix4 <- cbind(col1, col2)
print("Matrix 4 (using cbind):")
print(matrix4)

# 2. Accessing Matrix Elements
print(matrix2[2, 3])
print(matrix2[2, ])
print(matrix1[, 3])

# 3. Modifying Matrices
matrix1[1, 1] <- 10
matrix2[2, ] <- c(7, 8, 9)

# 4. Matrix Arithmetic
matrix_add <- matrix1 + matrix(9:1, 3, 3)
matrix_mult <- matrix1 * 2
matrix_dot <- matrix1 %*% t(matrix1)

# 5. Matrix Functions
t(matrix1)
rowSums(matrix1)
colSums(matrix1)
rowMeans(matrix1)
colMeans(matrix1)

# 6. Logical Operations
matrix1 > 5

# 7. Binding Matrices
matrix_combined <- cbind(matrix1, matrix2[, 1])

# 8. Diagonal and Identity Matrices
diag_matrix <- diag(1, nrow = 3, ncol = 3)
custom_diag <- diag(c(1, 2, 3))

# 9. Matrix Subsetting
sub_matrix <- matrix1[1:2, 2:3]

# 10. Matrix Operations
det_matrix <- det(matrix1)
inv_matrix <- solve(diag_matrix)

# 11. Checking Properties
dim(matrix1)
nrow(matrix1)
ncol(matrix1)
```

Result:

The matrix operations, including creation, modification, arithmetic, and analysis using R functions like `matrix()`, `rbind()`, `cbind()`, and `det()`, were successfully executed. Key results such as matrix dimensions, subsetting, and determinant calculation were verified.

5. DataFrames in R

Aim:

To learn the creation, manipulation, and analysis of DataFrames in R, including operations like accessing, filtering, sorting, and handling missing data.

Procedure:

1. **Creating a DataFrame**: Use the `data.frame()` function to create a structured table of data.
2. **Accessing Data**: Retrieve specific rows, columns, or cells using indexing and column names.
3. **Adding a New Column**: Append a column with the `$` operator.
4. **Adding a New Row**: Add rows using the `rbind()` function.
5. **Removing a Column**: Use the `$` operator to assign `NULL` to a column for removal.
6. **Summary Statistics**: Use functions like `summary()` to analyze columns.
7. **Handling Missing Data**: Introduce missing values, identify them using `is.na()`, and replace them with computed values.
8. **Filtering Data**: Filter rows using the `subset()` function.
9. **Sorting Data**: Sort rows using the `order()` function.
10. **Saving and Loading Data**: Save the DataFrame to a file using `write.csv()` and load it back with `read.csv()`.

Program:

```
# Step 1: Creating a DataFrame
students <- data.frame(
  StudentID = 1:5,
  Name = c("Alice", "Bob", "Charlie", "David", "Eve"),
  Age = c(20, 22, 23, 21, 22),
  Marks = c(85, 90, 78, 88, 95),
  Passed = c(TRUE, TRUE, TRUE, TRUE, TRUE)
)
print("Step 1: DataFrame Created")
print(students)

# Step 2: Accessing Data
print("\nStep 2: Accessing Data")
print("Access the first row:")
print(students[1, ])

print("Access the 'Name' column:")
print(students$Name)

print("Access specific data (Marks of 3rd student):")
print(students[3, "Marks"])

# Step 3: Adding a New Column
print("\nStep 3: Adding a New Column")
students$Grade <- c("A", "A+", "B", "A", "A+")
print("DataFrame after adding 'Grade' column:")
```

```
print(students)

# Step 4: Adding a New Row
print("\nStep 4: Adding a New Row")
new_row <- data.frame(StudentID = 6, Name = "Frank", Age = 23, Marks = 80, Passed = TRUE,
Grade = "B+")
students <- rbind(students, new_row)
print("DataFrame after adding a new row:")
print(students)

# Step 5: Removing a Column
print("\nStep 5: Removing a Column")
students$Passed <- NULL
print("DataFrame after removing the 'Passed' column:")
print(students)

# Step 6: Summary Statistics
print("\nStep 6: Summary Statistics")
print("Summary of the 'Marks' column:")
print(summary(students$Marks))

# Step 7: Handling Missing Data
print("\nStep 7: Handling Missing Data")
students$Marks[4] <- NA  # Introduce a missing value
print("DataFrame with a missing value:")
print(students)

print("Replacing missing values with the column mean:")
students$Marks[is.na(students$Marks)] <- mean(students$Marks, na.rm = TRUE)
print(students)

# Step 8: Filtering Data
print("\nStep 8: Filtering Data")
print("Filter students with Marks > 85:")
filtered_students <- subset(students, Marks > 85)
print(filtered_students)

# Step 9: Sorting Data
print("\nStep 9: Sorting Data")
print("Sort students by Age in descending order:")
sorted_students <- students[order(-students$Age), ]
print(sorted_students)

# Step 10: Saving and Loading Data
print("\nStep 10: Saving and Loading Data")
write.csv(students, "students.csv", row.names = FALSE)
print("DataFrame saved to 'students.csv'.")
loaded_students <- read.csv("students.csv")
print("DataFrame loaded from 'students.csv':")
print(loaded_students)
```

Result:

The data frame was successfully created, manipulated, and analyzed using various operations like adding/removing columns, filtering, and sorting. Missing data was handled effectively, and the DataFrame was saved and loaded from a CSV file.

6. Descriptive Statistics in R

Aim:

To calculate and interpret descriptive statistics (mean, median, mode, variance, standard deviation, and summary statistics) for a dataset in R.

Procedure

1. Import or create a dataset in R.
2. Compute basic descriptive statistics for numeric columns in the dataset:
    - Mean, Median, Mode, Variance and Standard Deviation
3. Generate summary statistics for the entire dataset.
4. Visualize the data using histograms and boxplots to understand the distribution and variability.

Program:

```
# Step 1: Load or Create a Dataset
students <- data.frame(
  StudentID = 1:10,
  Marks = c(78, 85, 62, 90, 88, 75, 80, 68, 95, 84),
  Age = c(20, 21, 20, 22, 21, 20, 22, 19, 23, 21)
)
print("Dataset:")
print(students)

# Step 2: Compute Descriptive Statistics
# Mean of Marks and Age
mean_marks <- mean(students$Marks)
mean_age <- mean(students$Age)
print(paste("Mean Marks:", mean_marks))
print(paste("Mean Age:", mean_age))

# Median of Marks and Age
median_marks <- median(students$Marks)
median_age <- median(students$Age)
print(paste("Median Marks:", median_marks))
print(paste("Median Age:", median_age))

# Mode Function
get_mode <- function(x) {
  uniq_vals <- unique(x)
  uniq_vals[which.max(tabulate(match(x, uniq_vals)))]
}

# Mode of Marks and Age
mode_marks <- get_mode(students$Marks)
mode_age <- get_mode(students$Age)
print(paste("Mode Marks:", mode_marks))
print(paste("Mode Age:", mode_age))
```

```
# Variance of Marks
variance_marks <- var(students$Marks)
print(paste("Variance of Marks:", variance_marks))

# Standard Deviation of Marks
std_dev_marks <- sd(students$Marks)
print(paste("Standard Deviation of Marks:", std_dev_marks))

# Step 3: Summary Statistics
summary_stats <- summary(students)
print("Summary Statistics:")
print(summary_stats)

# Step 4: Visualization
# Histogram for Marks
hist(
  students$Marks,
  main = "Histogram of Marks",
  xlab = "Marks",
  col = "lightblue",
  border = "black"
)

# Boxplot for Marks
boxplot(
  students$Marks,
  main = "Boxplot of Marks",
  ylab = "Marks",
  col = "lightgreen"
)
```

Result:

The program successfully computes descriptive statistics (mean, median, mode, variance, standard deviation) for the dataset and visualizes the data using a histogram and boxplot.

7. Data Preprocessing in R

Aim

To preprocess a dataset by handling missing data, encoding categorical data, splitting the dataset into training and test sets, and performing feature scaling.

Procedure

1. **Import the Dataset**:
   Read the dataset `purchase.csv` which contains customer information such as `Age`, `Salary`, `Country`, and `Purchased`.
2. **Handle Missing Data**:
   - Identify missing values in the `Age` and `Salary` columns.
   - Replace missing values with the average (mean) of the respective columns.
3. **Encode Categorical Data**:
   - Convert the categorical variable `Country` into numerical factors.
   - Convert the `Purchased` column into binary factors (0 for "no" and 1 for "yes").
   - Replace any missing values in the `Purchased` column with 0.
4. **Split the Dataset into Training and Test Sets**:
   - Use an 80:20 ratio for splitting the dataset.
   - Use the `caTools` library for random sampling.
5. **Feature Scaling**:
   - Apply feature scaling (standardization) to the numerical columns (`Age` and `Salary`) in both the training and test sets.

Program :

```
# Step 1: Importing the Dataset
data <- read.csv("purchase.csv")
print("Dataset:")
print(data)

# Step 2: Handling the Missing Data
# Identify missing values
is.na(data$Age)  # Finding missing values in the Age column
sum(is.na(data$Age))  # Counting missing values in the Age column

# Replace missing values in Age column with the mean
data$Age <- ifelse(is.na(data$Age),
                   ave(data$Age, FUN = function(x) mean(x, na.rm = TRUE)),
                   data$Age)

# Replace missing values in Salary column with the mean
data$Salary <- ifelse(is.na(data$Salary),
                      ave(data$Salary, FUN = function(x) mean(x, na.rm = TRUE)),
                      data$Salary)

# Step 3: Encoding Categorical Data
```

```
# Encode Country as numerical factors
data$Country <- factor(data$Country,
                       levels = c('India', 'Srilanka', 'Pakistan'),
                       labels = c(1.0, 2.0, 3.0))

# Encode Purchased as binary factors
data$Purchased <- factor(data$Purchased,
                         levels = c('No', 'Yes'),
                         labels = c(0.0, 1.0))
# Replace any missing Purchased values with 0
data$Purchased[is.na(data$Purchased)] <- 0
data$Purchased <- as.factor(data$Purchased)

# Step 4: Splitting the Dataset into Training and Test Sets
library(caTools)
set.seed(123)
split <- sample.split(data$Purchased, SplitRatio = 0.8)

# Create training and test sets
training_set <- subset(data, split == TRUE)
test_set <- subset(data, split == FALSE)

# Step 5: Feature Scaling
training_set[, c(3, 4)] <- scale(training_set[, c(3, 4)])
test_set[, c(3, 4)] <- scale(test_set[, c(3, 4)])

print("Training Set:")
print(training_set)
print("Test Set:")
print(test_set)
```

Sample `purchase.csv` File

	Custome	Counti	A	Sala	Purchas
1	India	25	50000	Yes	
2	Srilanka	30	NA	No	
3	Pakistan	35	60000	Yes	
4	India	NA	55000	Yes	
5	Srilanka	27	52000	No	
6	Pakistan	NA	58000	Yes	
7	India	32	62000	Yes	
8	Pakistan	40	50000	No	
9	Srilanka	29	48000	Yes	
10	India	38	55000	No	

Result:

The program successfully preprocesses the dataset by handling missing values, encoding categorical data, splitting it into training and test sets, and applying feature scaling to the `Age` and `Salary` columns.

8. Outlier Detection in the `mtcars` Dataset

Aim:
To detect and remove outliers using two different methods – Interquartile Range (IQR) and Z-Score Method – and compare their effectiveness in cleaning the `mtcars` dataset.

Procedure:

Step 1: Load and Explore the Dataset
- Load the `mtcars` dataset and inspect its structure.

Step 2: Visualize the Initial Data Distribution
- Plot histograms and boxplots to identify outliers in:
  - MPG (Miles Per Gallon)
  - Weight (wt)

Step 3: Implement IQR Method for Outlier Detection
- Compute Q1, Q3, and IQR.
- Identify and remove outliers in `mpg` and `wt`.

Step 4: Implement Z-Score Method for Outlier Detection
- Compute Z-scores for `mpg` and `wt`.
- Identify and remove outliers based on a threshold of |Z| > 3.

Step 5: Compare the Results of IQR and Z-Score Methods
- Print outlier values detected by each method.
- Compare dataset dimensions before and after outlier removal.

Program:

```
# Step 1: Load the Dataset
data <- mtcars

# Step 2: Visualize the Initial Distribution
hist(data$mpg, main = "Histogram of MPG (Before Cleaning)", xlab = "MPG", col = "lightblue")
boxplot(data$mpg, main = "Boxplot of MPG (Before Cleaning)", ylab = "MPG", col = "lightblue")

hist(data$wt, main = "Histogram of Weight (Before Cleaning)", xlab = "Weight", col = "lightcoral")
boxplot(data$wt, main = "Boxplot of Weight (Before Cleaning)", ylab = "Weight", col = "lightcoral")

# Step 3: Define IQR Method for Outlier Detection
iqr_detect <- function(x) {
  q <- quantile(x, probs = c(0.25, 0.75))  # Compute Q1 and Q3
  iqr <- q[2] - q[1]  # Compute IQR
  lower <- q[1] - 1.5 * iqr  # Lower Bound
  upper <- q[2] + 1.5 * iqr  # Upper Bound
  return(x[x < lower | x > upper])  # Return outliers
}

# Detect Outliers using IQR
mpg_outliers_iqr <- iqr_detect(data$mpg)
wt_outliers_iqr <- iqr_detect(data$wt)

print("Outliers in MPG using IQR:")
print(mpg_outliers_iqr)
```

```
print("Outliers in Weight using IQR:")
print(wt_outliers_iqr)

# Remove Outliers using IQR
cleaned_data_iqr <- data[!(data$mpg %in% mpg_outliers_iqr | data$wt %in% wt_outliers_iqr), ]

# Step 4: Define Z-Score Method for Outlier Detection
zscore_detect <- function(x, threshold = 3) {
  z_scores <- (x - mean(x)) / sd(x)  # Compute Z-scores
  return(x[abs(z_scores) > threshold])  # Return outliers with |Z| > 3
}

# Detect Outliers using Z-Score
mpg_outliers_z <- zscore_detect(data$mpg)
wt_outliers_z <- zscore_detect(data$wt)

print("Outliers in MPG using Z-Score:")
print(mpg_outliers_z)

print("Outliers in Weight using Z-Score:")
print(wt_outliers_z)

# Remove Outliers using Z-Score
cleaned_data_zscore <- data[!(data$mpg %in% mpg_outliers_z | data$wt %in% wt_outliers_z), ]

# Step 5: Compare Dataset Size Before and After Cleaning
print("Original Dataset Size:")
print(dim(data))

print("Dataset Size after IQR Outlier Removal:")
print(dim(cleaned_data_iqr))

print("Dataset Size after Z-Score Outlier Removal:")
print(dim(cleaned_data_zscore))

# Step 6: Visualize the Data After Outlier Removal
par(mfrow = c(2,2))  # Arrange plots in 2x2 grid

# MPG Visualizations
boxplot(cleaned_data_iqr$mpg, main = "Boxplot of MPG (After IQR Cleaning)", ylab = "MPG", col = "lightgreen")
boxplot(cleaned_data_zscore$mpg, main = "Boxplot of MPG (After Z-Score Cleaning)", ylab = "MPG", col = "lightgoldenrod")

# Weight Visualizations
boxplot(cleaned_data_iqr$wt, main = "Boxplot of Weight (After IQR Cleaning)", ylab = "Weight", col = "lightgreen")
boxplot(cleaned_data_zscore$wt, main = "Boxplot of Weight (After Z-Score Cleaning)", ylab = "Weight", col = "lightgoldenrod")
```

Result:

The program successfully detects and handles outliers in the `mpg` and `wt` variables of the `mtcars` dataset using histograms, boxplots, and IQR-based calculations. The cleaned data is visualized, and dimensional changes in the dataset are validated.

9. Linear Regression in R

Aim:
To implement Linear Regression using R for predicting weight based on height and evaluate the model using RMSE and R-squared metrics.

Procedure:

1. Define predictor (`x`: height) and response (`y`: weight) variables.
2. Apply the `lm()` function to create a linear regression model.
3. Predict the weight of a person with height 170 cm using the model.
4. Visualize the regression using a scatter plot with a fitted line.
5. Load the `women` dataset and check its structure.
6. Split the dataset into training (80%) and testing (20%) sets.
7. Fit a linear regression model using the training data.
8. Evaluate the model using RMSE and R-squared values.
9. Visualize the regression using the `ggplot2` library.

Program:

```
# Step 1: Define Predictor and Response Variables
x <- c(151, 174, 138, 186, 128, 136, 179, 163, 152, 131)  # Heights
y <- c(63, 81, 56, 91, 47, 57, 76, 72, 62, 48)           # Weights

# Step 2: Apply Linear Regression
relation <- lm(y ~ x)
print(relation)
summary(relation)

# Step 3: Predict Weight for a Given Height (170 cm)
a <- data.frame(x = 170)
result <- predict(relation, a)
print(paste("weight for height ",a,"is:",result))

# Step 4: Visualize the Regression Line
png(file = "linear_regression.png")
plot(y, x, col = "blue", main = "Height & Weight Regression",
    abline(lm(x ~ y)), cex = 1.3, pch = 16, xlab = "Weight in Kg", ylab = "Height in cm")
dev.off()

# Step 5: Load and Explore the Dataset
data <- women
str(data)

# Step 6: Split Data into Training and Testing Sets
set.seed(123)
trainIndex <- sample(1:nrow(data), 0.8 * nrow(data))
trainData <- data[trainIndex, ]
testData <- data[-trainIndex, ]
```

```
# Step 7: Fit the Linear Regression Model
model <- lm(weight ~ ., data = trainData)
summary(model)

# Step 8: Make Predictions on Test Set
predictions <- predict(model, testData)

# Step 9: Evaluate the Model using RMSE and R-Squared
# install.packages("Metrics")
library(Metrics)
rmse_value <- rmse(predictions, testData$weight)

rSquared <- function(predicted, actual) {
  1 - sum((actual - predicted) ^ 2) / sum((actual - mean(actual)) ^ 2)
}

rSquared_value <- rSquared(predictions, testData$weight)

# Print the Evaluation Metrics
cat("RMSE:", rmse_value, "\n")
cat("R-squared:", rSquared_value, "\n")

# Step 10: Visualize the Regression Model using ggplot2
library(ggplot2)
ggplot(data, aes(x = height, y = weight)) +
  geom_point() +
  geom_smooth(method = "lm") +
  xlab("Height (cm)") +
  ylab("Weight (kg)") +
  ggtitle("Linear Regression: Height vs Weight in Women Dataset")
```

Result:

The program successfully implements linear regression for predicting weight based on height, evaluates model performance using RMSE and R-squared metrics, and visualizes the regression results.

10. Implementation of Naïve Bayes

Aim:

To implement the Naïve Bayes Classifier in R using the `e1071` package and predict whether a new customer will buy a computer based on given attributes.

Procedure:

DATA:

Age	Income	Student	Credit_Rating	Buys_computer
<=30	High	No	Fair	No
<=30	High	No	Excellence	No
31...40	High	No	Fair	Yes
>40	Medium	No	Fair	Yes
>40	Low	Yes	Fair	Yes
>40	Low	Yes	Excellence	No
31...40	Low	Yes	Excellence	Yes
<=30	Medium	No	Fair	No
<=30	Low	Yes	Fair	Yes
>40	Medium	Yes	Fair	Yes
<=30	Low	Yes	Fair	Yes
31...40	Medium	No	Excellence	Yes
31...40	High	Yes	Fair	Yes
>40	Medium	No	Excellence	No

Step 1: Install Required Package

Install the `e1071` package if not already installed using: `install.packages("e1071")`
Load the package using: `library(e1071)`

Step 2: Load and Prepare the Dataset

- Create a **data frame** with the given attributes:
  - **Age**
  - **Income**
  - **Student**
  - **Credit Rating**
  - **Buys_computer** (Target variable)
- Convert categorical variables into **factors** for proper handling.

Step 3: Train the Naïve Bayes Model

- Use the `naiveBayes()` function from the `e1071` package to train the model.
- The formula used is: Buys_computer~Age+Income+Student+Credit_rating

Step 4: Compute Prior and Conditional Probabilities

- **Prior Probability** is calculated as:
  P(Yes)=9/14,P(No)=5/14
- **Likelihood Probabilities** for each attribute are calculated using frequency counts.

Step 5: Predict for a New Customer

- Define a **new customer** with specific attributes:
  - Age = **<=30**
  - Income = **Medium**
  - Student = **Yes**
  - Credit Rating = **Fair**
- Use `predict()` to classify the new data point.

Step 6: Compute Posterior Probabilities

- Using **Bayes' Theorem**, the probability of buying a computer (`Yes`) is computed as:
  $$P(Yes|x) \propto P(Yes) \times P(Age|Yes) \times P(Income|Yes) \times P(Student|Yes) \times P(Credit_Rating|Yes)$$
- Normalize the probabilities to obtain a final prediction.

Program:

```
# Install and load the required package
install.packages("e1071")
library(e1071)

# Load required library for data splitting
install.packages("caTools")
library(caTools)

# Define the dataset
data <- data.frame(
  Age = factor(c("<=30", "<=30", "31...40", ">40", ">40", ">40", "31...40", "<=30", "<=30", ">40", "<=30", "31...40", "31...40", ">40")),
  Income = factor(c("High", "High", "High", "Medium", "Low", "Low", "Low", "Medium", "Low", "Medium", "Low", "Medium", "High", "Medium")),
  Student = factor(c("No", "No", "No", "No", "Yes", "Yes", "Yes", "No", "Yes", "Yes", "Yes", "No", "Yes", "No")),
 Credit_rating = factor(c("Fair", "Excellence", "Fair", "Fair", "Fair", "Excellence", "Excellence", "Fair", "Fair", "Fair", "Fair", "Excellence", "Fair", "Excellence")),
  Buys_computer = factor(c("No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No", "Yes", "Yes", "Yes", "Yes", "Yes", "No"))
)

# Splitting the dataset into Training (70%) and Test (30%) sets
set.seed(123)  # For reproducibility
split <- sample.split(data$Buys_computer, SplitRatio = 0.7)
train_set <- subset(data, split == TRUE)
test_set <- subset(data, split == FALSE)

# Train the Naive Bayes model using the training set
model <- naiveBayes(Buys_computer ~ ., data = train_set)
print(model)

# Make predictions on the test set
test_predictions <- predict(model, newdata = test_set)
```

```
# Evaluate model performance
conf_matrix <- table(Predicted = test_predictions, Actual = test_set$Buys_computer)
print("Confusion Matrix:")
print(conf_matrix)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
print(paste("Model Accuracy:", round(accuracy * 100, 2), "%"))

# Define a new customer for prediction
new_customer <- data.frame(
  Age = factor("<=30", levels = levels(data$Age)),
  Income = factor("Medium", levels = levels(data$Income)),
  Student = factor("Yes", levels = levels(data$Student)),
  Credit_rating = factor("Fair", levels = levels(data$Credit_rating))
)

# Predict for new customer
prediction <- predict(model, newdata = new_customer)
print(paste("Prediction for new customer:", prediction))

# Compute probabilities
probabilities <- predict(model, newdata = new_customer, type = "raw")
print("Probabilities for new customer:")
print(probabilities)
```

Result:
The Naïve Bayes model predicted that the new customer **will buy a computer (Yes)** with a probability of **0.66%**.

11. K-Means Clustering on Customer Data for Segmentation

Aim:

To apply K-Means Clustering in R to segment customers based on Annual Income and Spending Score.

Procedure:

Data: customers.csv

CustomerID	Genre	Age	Annual_Income	Spending_Score
1	Male	19	15	39
2	Male	21	15	81
3	Female	20	16	6
4	Female	23	16	77
5	Female	31	17	40
6	Female	22	17	76

Step 1: Install and Load Required Libraries
Before performing K-Means Clustering, we need to install and load the necessary R packages:

- `ggplot2` → Used for visualizing data and clusters.
- `factoextra` → Used for determining the optimal number of clusters (K) using the Elbow Method.
- `cluster` → Provides functions for clustering techniques, including K-Means.

Step 2: Load and Explore the Dataset

- Read the Mall Customers dataset (`customers.csv`) into R.
- Understand its structure using functions like `head()`, `str()`, and `summary()`.
- Since K-Means requires numerical features, we select only Annual_Income and Spending_Score for clustering.
- The `CustomerID` column is irrelevant for clustering, and `Genre` is categorical, so both will be ignored.

Step 3: Determine the Optimal Number of Clusters (K)

- Why do we need this step?
  - K-Means requires us to predefine K (number of clusters), so we must determine the best value for K.
- How do we find the best K?
  - We use the Elbow Method, which plots the Within-Cluster Sum of Squares (WCSS) vs. Number of Clusters.
  - The optimal K is chosen at the point where the WCSS curve bends like an "elbow".

Step 4: Apply K-Means Clustering

- Based on the Elbow Method, we choose an optimal K (e.g., K=5).
- We apply the K-Means clustering algorithm using the `kmeans()` function in R.
- The algorithm iteratively assigns data points to the nearest cluster centroids and updates the centroids until they stabilize.

Step 5: Visualize the Final Clusters

- To better understand the segmentation, we plot a scatter plot of Annual Income vs. Spending Score, where each cluster is colored differently, and centroids are marked in red.

Step 6: Analyze the Cluster Results

- Print the cluster centers (final centroids).
- Display the cluster assignments for each customer.

Program:

```
# Step 1: Install and Load Required Libraries
install.packages("ggplot2")       # For visualization
install.packages("factoextra")    # For evaluating K
library(ggplot2)
library(factoextra)

# Step 2: Load the Dataset
data <- read.csv("customers.csv")
# View the first few rows
head(data)

# Step 3: Select Relevant Features for Clustering
customer_data <- data[, c("Annual_Income", "Spending_Score")]

# Step 4: Determine the Optimal K using Elbow Method
fviz_nbclust(customer_data, kmeans, method = "wss") +
  ggtitle("Elbow Method for Optimal K")

# Step 5: Apply K-Means Clustering (Choosing K=5)
set.seed(123)
k_value <- 5
kmeans_result <- kmeans(customer_data, centers = k_value, nstart = 25)

# Step 6: Assign Clusters to Data
customer_data$Cluster <- as.factor(kmeans_result$cluster)

# Step 7: Visualize the Clusters
ggplot(customer_data, aes(x = Annual_Income, y = Spending_Score, color = Cluster)) +
  geom_point(size = 3) +
  geom_point(data = as.data.frame(kmeans_result$centers), aes(x = Annual_Income, y =
Spending_Score), color = "red", size = 5, shape = 8) +
  ggtitle("K-Means Clustering of Customers") +
  xlab("Annual Income") +
  ylab("Spending Score") +
  theme_minimal()

# Step 8: Print Cluster Centers
print("Cluster Centers:")
print(kmeans_result$centers)

# Step 9: Print Cluster Assignments
print("Cluster Assignments:")
print(kmeans_result$cluster)
```

Result:

The K-Means clustering model successfully segmented the customers into 5 distinct clusters based on Annual Income and Spending Score. The optimal number of clusters was determined using the Elbow Method. The final cluster assignments and centroids were identified, and the clusters were visualized using a scatter plot.

12. K-Medoids Clustering on the Iris Dataset (a)

Aim:

To perform K-Medoids clustering on the Iris dataset.

Procedure:

Step 1: Install and Load Required Libraries

- cluster → Used for K-Medoids clustering (pam()).
- ggplot2 → Used for data visualization.
- factoextra → Used for finding the optimal K and silhouette analysis.

Step 2: Load and Explore the Dataset

- Use the iris dataset, which contains measurements of three flower species.
- Select two numeric columns (Petal.Length and Petal.Width) for clustering.

Step 3: Determine the Optimal Number of Clusters (K)

- Use the Gap Statistic Method to find the best K.

Step 4: Apply K-Medoids Clustering (Using Optimal K)

- Use the pam() function (Partitioning Around Medoids).

Step 5: Assign Cluster Labels Based on Species Names

- Map the cluster numbers (1, 2, 3) to actual species (Setosa, Versicolor, Virginica).

Step 6: Compute and Visualize Silhouette Scores

- Calculate silhouette scores for each data point.
- Display Silhouette Visualization to check how well-separated the clusters are.

Step 7: Compare Clusters with Actual Species Labels

- Validate how well the clustering matches the actual species labels.

Program :

```
# Step 1: Install and Load Required Libraries
install.packages("cluster")       # For K-Medoids (PAM algorithm)
install.packages("ggplot2")       # For visualization
install.packages("factoextra")    # For optimal K selection & silhouette analysis
install.packages("cluster")       # For Gap Statistic Method
library(cluster)
library(ggplot2)
library(factoextra)

# Step 2: Load the Iris Dataset
data <- iris  # Load dataset
head(data)    # View first few rows

# Step 3: Select Features for Clustering (Petal Length & Petal Width for better separation)
iris_data <- data[, c("Petal.Length", "Petal.Width")]

# Step 4: Visualize the Raw Data (Before Clustering)
ggplot(iris_data, aes(x = Petal.Length, y = Petal.Width)) +
  geom_point(color = "blue", size = 3) +
  ggtitle("Before Clustering: Raw Data") +
  xlab("Petal Length") +
  ylab("Petal Width") +
  theme_minimal()
```

```
# Step 5: Determine the Optimal K using Gap Statistic Method
set.seed(123)
gap_stat <- clusGap(iris_data, FUN = pam, K.max = 10, B = 50)
fviz_gap_stat(gap_stat)  # Visualization of Gap Statistic results

# Extract the best K (should be 3 for Iris dataset)
optimal_k <- maxSE(gap_stat$Tab[, "gap"], gap_stat$Tab[, "SE.sim"])
print(paste("Optimal K found by Gap Statistic Method:", optimal_k))

# Step 6: Apply K-Medoids Clustering using the detected K
kmedoids_result <- pam(iris_data, k = optimal_k)

# Step 7: Assign Cluster Labels Based on Species
# Mapping clusters to actual species
cluster_labels <- c("Setosa", "Versicolor", "Virginica")

# Assign human-readable labels
iris_data$Cluster <- factor(kmedoids_result$cluster, labels = cluster_labels)

# Step 8: Compute Silhouette Scores for Validation
silhouette_scores <- silhouette(kmedoids_result$clustering, dist(iris_data[, 1:2]))

# Step 9: Visualize Silhouette Scores for Each Cluster
fviz_silhouette(silhouette_scores) +
  ggtitle("Silhouette Scores for K-Medoids Clusters")

# Step 10: Visualize the Clusters (After Clustering)
ggplot(iris_data, aes(x = Petal.Length, y = Petal.Width, color = Cluster)) +
  geom_point(size = 3) +
    geom_point(data = as.data.frame(kmedoids_result$medoids), aes(x = Petal.Length, y =
Petal.Width), color = "red", size = 5, shape = 8) +
  ggtitle(paste("After Clustering: K-Medoids (K =", optimal_k, ")")) +
  xlab("Petal Length") +
  ylab("Petal Width") +
  theme_minimal()

# Step 11: Compare Clusters with Actual Species
iris_data$Actual_Species <- data$Species  # Add actual species for comparison
confusion_matrix <- table(Predicted_Cluster = iris_data$Cluster, Actual_Species =
iris_data$Actual_Species)
print("Comparison of Predicted Clusters with Actual Species:")
print(confusion_matrix)

# Step 12: Print Medoids (Cluster Centers)
print("Medoid Centers:")
print(kmedoids_result$medoids)

# Step 13: Print Cluster Assignments with Labels
print("Cluster Assignments:")
print(iris_data[, c("Petal.Length", "Petal.Width", "Cluster")])  # Show actual cluster labels

# Step 14: Compute Average Silhouette Score
avg_silhouette <- mean(silhouette_scores[, "sil_width"])
print(paste("Average Silhouette Score:", round(avg_silhouette, 3)))
```

Result :

The Gap Statistic Method correctly detected K = 3, and K-Medoids successfully classified the Iris dataset into 3 clusters, aligning with the three actual species.

12. K-Medoids Clustering on the Iris Dataset (b)

Aim:

To implement K-Medoids clustering on the Iris dataset, where the number of clusters (K) is fixed at 3, corresponding to the three species: Setosa, Versicolor, and Virginica.

Procedure:

Step 1: Install and Load Required Libraries

- cluster → Used for K-Medoids clustering (pam()).
- ggplot2 → Used for data visualization.
- factoextra → Used for plotting clusters.

Step 2: Load and Explore the Dataset

- Use the iris dataset, which contains measurements of three flower species.
- Select two numeric columns (Petal.Length and Petal.Width) for clustering.

Step 3: Set K = 3 and Apply K-Medoids Clustering

- Use the Partitioning Around Medoids (PAM) algorithm to cluster the data into K = 3 clusters.

Step 4: Assign Cluster Labels Based on Species Names

- Map the cluster numbers (1, 2, 3) to actual species (Setosa, Versicolor, Virginica).

Step 5: Compare Clusters with Actual Species Labels

- Validate how well the clustering matches the actual species labels using a confusion matrix.

Step 6: Visualize the Clusters

- Display scatter plots of clusters before and after clustering.

Program:

```
# Step 1: Install and Load Required Libraries
install.packages("cluster")       # For K-Medoids (PAM algorithm)
install.packages("ggplot2")       # For visualization
install.packages("factoextra")    # For plotting clusters
library(cluster)
library(ggplot2)
library(factoextra)

# Step 2: Load the Iris Dataset
data <- iris  # Load dataset
head(data)    # View first few rows

# Step 3: Select Features for Clustering (Petal Length & Petal Width for better separation)
iris_data <- data[, c("Petal.Length", "Petal.Width")]

# Step 4: Visualize the Raw Data (Before Clustering)
ggplot(iris_data, aes(x = Petal.Length, y = Petal.Width)) +
  geom_point(color = "blue", size = 3) +
  ggtitle("Before Clustering: Raw Data") +
  xlab("Petal Length") +
```

```
  ylab("Petal Width") +
  theme_minimal()

# Step 5: Apply K-Medoids Clustering using K = 3
k <- 3  # Fixed K
kmedoids_result <- pam(iris_data, k = k)

# Step 6: Assign Cluster Labels Based on Species
# Mapping clusters to actual species
cluster_labels <- c("Setosa", "Versicolor", "Virginica")

# Assign human-readable labels
iris_data$Cluster <- factor(kmedoids_result$cluster, labels = cluster_labels)

# Step 7: Visualize the Clusters (After Clustering)
ggplot(iris_data, aes(x = Petal.Length, y = Petal.Width, color = Cluster)) +
  geom_point(size = 3) +
    geom_point(data = as.data.frame(kmedoids_result$medoids), aes(x = Petal.Length, y =
Petal.Width), color = "red", size = 5, shape = 8) +
  ggtitle(paste("After Clustering: K-Medoids (K =", k, ")")) +
  xlab("Petal Length") +
  ylab("Petal Width") +
  theme_minimal()

# Step 8: Compare Clusters with Actual Species
iris_data$Actual_Species <- data$Species  # Add actual species for comparison
confusion_matrix <- table(Predicted_Cluster = iris_data$Cluster, Actual_Species =
iris_data$Actual_Species)
print("Comparison of Predicted Clusters with Actual Species:")
print(confusion_matrix)

# Step 9: Print Medoids (Cluster Centers)
print("Medoid Centers:")
print(kmedoids_result$medoids)

# Step 10: Print Cluster Assignments with Labels
print("Cluster Assignments:")
print(iris_data[, c("Petal.Length", "Petal.Width", "Cluster")])  # Show actual cluster labels
```

Result

The K-Medoids algorithm successfully classified the Iris dataset into 3 clusters, aligning with the three actual species.

13. K-Nearest Neighbors (KNN) Classification with Optimal K Selection

Aim:

To implement K-Nearest Neighbors (KNN) classification on the Breast Cancer Wisconsin Dataset, use cross-validation to find the best K value, and evaluate the model's performance.

Procedure:

Step 1: Install and Load Required Libraries

- class → Used for the KNN classification model.
- caret → Used for data partitioning, cross-validation, and model tuning.

Step 2: Load and Explore the Dataset

- Read the dataset from breast-cancer-wisconsin.data.txt.
- Assign column names.
- Convert the diagnosis class (2 = Benign, 4 = Malignant) into a factor.

Step 3: Handle Missing Values

- Remove rows with missing values (?).

Step 4: Preprocess the Data

- Normalize the numeric features between 0 and 1.
- Split the dataset into training (80%) and testing (20%) sets.

Step 5: Perform Grid Search to Find Optimal K

- Use 10-fold cross-validation to determine the best K value.

Step 6: Implement KNN with Optimal K and Evaluate

- Train the model using the best K value.
- Predict the tumor classification in the test set.
- Compute confusion matrix and accuracy.

Program :

```
############# KNN ############

# Step 1: Load necessary libraries
install.packages("class")
install.packages("caret")
library(class)
library(caret)

# Step 2: Load the dataset
data <- read.table("breast-cancer-wisconsin.data.txt", sep = ",", header = FALSE, na.strings = "?")
```

```
colnames(data) <- c("Sample_code_number", "Clump_Thickness", "Uniformity_of_Cell_Size",
                    "Uniformity_of_Cell_Shape", "Marginal_Adhesion", "Single_Epithelial_Cell_Size",
                    "Bare_Nuclei", "Bland_Chromatin", "Normal_Nucleoli", "Mitoses", "Class")

# Step 3: Handle missing values
data <- na.omit(data)

# Step 4: Drop the Sample_code_number column (not useful for classification)
data <- data[, -1]

# Remove column names
data=data[-1, ]

# Step 5: Convert all feature columns to numeric
data[, 1:9] <- lapply(data[, 1:9], as.numeric)

# Step 6: Normalize features (scaling between 0 and 1)
normalize <- function(x) { (x - min(x)) / (max(x) - min(x)) }
data[, 1:9] <- as.data.frame(lapply(data[, 1:9], normalize))

# Step 7: Convert 'Class' column to a factor
data$Class <- factor(data$Class, levels = c(2, 4), labels = c("Benign", "Malignant"))

# Step 8: Split the dataset into training and testing sets (80-20 split)
set.seed(123)  # For reproducibility
train_idx <- sample(1:nrow(data), size = 0.8 * nrow(data))
train_data <- data[train_idx, ]
test_data <- data[-train_idx, ]

# Step 9: Extract features and labels
train_features <- train_data[, -10]  # All columns except 'Class'
train_labels <- train_data[, 10]    # 'Class' column
test_features <- test_data[, -10]
test_labels <- test_data[, 10]

# Step 10: Grid search to tune k (number of neighbors) using cross-validation
control <- trainControl(method = "cv", number = 10)  # 10-fold cross-validation
k_values <- data.frame(k = seq(1, 20, by = 1))

# Perform grid search
knn_tuning <- train(
  x = train_features, y = train_labels,
  method = "knn",
  tuneGrid = k_values,
  trControl = control
)
```

```
# Step 11: Best value of k
cat("Optimal k value:", knn_tuning$bestTune$k, "\n")

# Step 12: Train the KNN model with the optimal k
k_optimal <- knn_tuning$bestTune$k
predictions <- knn(train = train_features, test = test_features, cl = train_labels, k = k_optimal)

# Step 13: Evaluate the model
confusion_matrix <- table(Predicted = predictions, Actual = test_labels)
print("Confusion Matrix:")
print(confusion_matrix)

# Step 14: Calculate accuracy
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Accuracy:", accuracy, "\n")
```

Result

The KNN classification model successfully classified the tumors as benign or malignant, achieving an accuracy of 96.35% using the optimal K value found by cross-validation.

14. Implementation of Support Vector Machine (SVM) for Loan Approval Prediction Using R

Aim:

To implement Support Vector Machine (SVM) classification in R using the Loan Dataset, predict whether a loan applicant will get approval, and evaluate the model's performance.

Procedure:

Step 1: Install and Load Required Libraries

- e1071 → Provides SVM functions.
  caret → Used for data partitioning and evaluation.
  ggplot2 → Used for visualization.

Step 2: Load and Explore the Dataset

- Read the dataset from "loan_dataset.csv".
  Check for missing values and remove them if needed.

Step 3: Encode Categorical Variables

- Convert categorical data into numeric values (Label Encoding).

Step 4: Preprocess the Data

- Remove irrelevant columns (Loan_ID, Gender).
  Split the dataset into training (95%) and testing (5%) sets using stratified sampling.

Step 5: Train the SVM Model

- Use a linear kernel for classification.
  Fit the training data to the SVM model.

Step 6: Make Predictions and Evaluate Model Performance

- Compute confusion matrix and accuracy score.

Step 7: Predict Loan Approval for New Customers

- Provide sample customer data to check if the loan is approved.

Program :

```
# Step 1: Install and Load Required Libraries
if (!require("e1071")) install.packages("e1071")
if (!require("caret")) install.packages("caret")
if (!require("ggplot2")) install.packages("ggplot2")
library(e1071)
library(caret)
library(ggplot2)

# Step 2: Load the Dataset
data <- read.csv("loan_dataset.csv", na.strings = "?")

# Step 3: Handle Missing Values (Robust Handling)
data <- data[complete.cases(data), ] # Remove rows with ANY NA

# Step 4: Convert Categorical Variables to Numeric
data$Loan_Status <- factor(ifelse(data$Loan_Status == "Y", 1, 0)) # Convert to Factor

# Corrected Dependents Conversion (Robust Handling)
data$Dependents <- as.character(data$Dependents)
data$Dependents[data$Dependents == "3+"] <- "4"
data$Dependents <- as.numeric(data$Dependents)

data$Married <- ifelse(data$Married == "Yes", 1, 0)
data$Self_Employed <- ifelse(data$Self_Employed == "Yes", 1, 0)
data$Education <- ifelse(data$Education == "Graduate", 1, 0)
```

```
data$Property_Area <- factor(data$Property_Area, levels = c("Rural", "Semiurban", "Urban"))
data$Property_Area <- as.numeric(data$Property_Area) - 1

# Step 5: Remove Unnecessary Columns
data <- subset(data, select = -c(Loan_ID, Gender))

# Step 6: Split the Data into Training (80%) and Testing (20%) Sets - Stratified
set.seed(42)
train_index <- createDataPartition(data$Loan_Status, p = 0.95, list = FALSE) # Stratified
train_data <- data[train_index, ]
test_data <- data[-train_index, ]

# Step 7: Extract Features and Labels
train_features <- subset(train_data, select = -Loan_Status)
train_labels <- train_data$Loan_Status
test_features <- subset(test_data, select = -Loan_Status)
test_labels <- test_data$Loan_Status

# Step 8: Train the SVM Model with a Linear Kernel
svm_model <- svm(Loan_Status ~ ., data = train_data, kernel = "linear")

# Step 9: Make Predictions on the Test Set
test_predictions <- predict(svm_model, test_features)

# Step 10: Evaluate the Model
conf_matrix <- table(Predicted = test_predictions, Actual = test_labels)
print("Confusion Matrix:")
print(conf_matrix)

# Step 11: Verify Confusion Matrix Consistency
total_predictions <- sum(conf_matrix)
expected_test_size <- length(test_labels)
cat("Total Predictions in Confusion Matrix:", total_predictions, "\n")
cat("Expected Test Set Size:", expected_test_size, "\n")

# Step 12: Compute Accuracy
accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix) * 100
cat("Model Accuracy:", round(accuracy, 2), "%\n")

# Step 13: Predict Loan Approval for New Customers
new_customer1 <- data.frame(Married = 1, Dependents = 1, Education = 1, Self_Employed = 1,
                            ApplicantIncome = 12841, CoapplicantIncome = 10968, LoanAmount = 349,
                            Loan_Amount_Term = 360, Credit_History = 1, Property_Area = 1)

new_customer2 <- data.frame(Married = 1, Dependents = 1, Education = 1, Self_Employed = 0,
                            ApplicantIncome = 300, CoapplicantIncome = 30000, LoanAmount = 39,
                            Loan_Amount_Term = 30, Credit_History = 0, Property_Area = 1)
# Make predictions
prediction1 <- predict(svm_model, new_customer1)
prediction2 <- predict(svm_model, new_customer2)

# Display Loan Status
cat("Prediction for New Customer 1: ", ifelse(prediction1 == 1, "Loan Approved", "Loan Rejected"), "\n")
cat("Prediction for New Customer 2: ", ifelse(prediction2 == 1, "Loan Approved", "Loan Rejected"), "\n")
```

Result:

The SVM model achieved an accuracy of 88.46% on the test dataset. The model successfully predicted loan approval for new customers, approving one and rejecting the other based on their financial details.

15. Implementation of the Apriori Algorithm for Association Rule Mining Using R

Aim:

To implement Association Rule Mining using the Apriori algorithm in R, extract meaningful patterns from transactional data, and analyze the relationships between items.

Procedure:

Step 1: Install and Load Required Libraries

`arules` → For association rule mining.

`arulesViz` → For rule visualization.

Step 2: Load the Dataset

Use the built-in Groceries dataset (from `arules`).

This dataset contains 9835 transactions with supermarket purchases.

Step 3: Inspect the Dataset

Check the structure and summary of the dataset.

View sample transactions.

Step 4: Apply the Apriori Algorithm

Set a minimum support and confidence threshold.

Extract frequent itemsets and generate rules.

Step 5: Display and Analyze the Rules

Show a summary of the generated rules.

Print the top 5 rules sorted by lift.

Step 6: Visualization of Association Rules

Use a scatter plot to analyze rule distribution.

Use a graph-based visualization to observe relationships.

Step 7: Experiment with Different Parameters

Modify support, confidence, and lift thresholds to see how the number of rules changes.

Step 8: Interpret the Results

Identify strong association rules that can be used for business insights.

Program :

```
# Step 1: Install and Load Required Libraries
if (!require("arules")) install.packages("arules", dependencies=TRUE)
if (!require("arulesViz")) install.packages("arulesViz", dependencies=TRUE)
library(arules)
library(arulesViz)

# Step 2: Load the Dataset
data("Groceries")  # Load built-in transactional dataset

# Step 3: Inspect the Dataset
summary(Groceries)  # Check dataset summary
inspect(Groceries[1:5])  # View first 5 transactions

# Step 4: Generate Association Rules using Apriori Algorithm
rules <- apriori(Groceries,
            parameter = list(supp = 0.01, conf = 0.2, minlen = 2))

# Step 5: Display Summary of Rules
summary(rules)
```

```
# Step 6: View the Top 5 Strongest Rules Sorted by Lift
inspect(sort(rules, by="lift")[1:5])

# Step 7: Visualization of Rules
plot(rules, method = "scatterplot", measure = c("support", "confidence"), shading = "lift")
plot(rules, method = "graph", control = list(type = "items"))

# Step 8: Experiment with Different Support and Confidence Thresholds
rules_high_conf <- apriori(Groceries,
                           parameter = list(supp = 0.02, conf = 0.5, minlen = 2))
inspect(sort(rules_high_conf, by="lift")[1:5])
```

Result

The Apriori algorithm successfully extracted association rules from the supermarket transaction dataset, identifying key purchasing patterns and item relationships.